

从零训 Code LLM · Part 4

RL 训练代码模型:从策略梯度到 GRPO、从单步生成到多步 debug

第一性推演 RL 用于 code generation(单步 bandit)与 code debug / SWE-agent(多步 MDP)。含 $J(\theta)$ 完整展开、大数定律、log-derivative trick、GRPO、advantage 加权的自生成序列 CE、LoRA。为下一步 Kaggle SWE-agent 竞赛打底。

脉络

- 1. RL 核心:去掉数学包装后就一句话
- 2. 目标 $J(\theta)$ 的完整展开(含策略概率、转移、 V)
- 3. 策略梯度:log-derivative trick + 大数定律(蒙特卡洛)
- 4. advantage: $A=Q-V$,单步退化成 r -baseline
- 5. Code generation:单步 bandit + GRPO(砍 critic)
- 6. policy loss = advantage 加权的自生成序列 CE
- 7. clip 与 KL 锚定(PPO 的 proximal)
- 8. 框架 vs 算法:RLVR / RLHF / GRPO / PPO
- 9. Code debug / SWE-agent:多步 MDP + GAE
- 10. state 表示:文本 / 执行图 / 变量值,与 LLM 的"心算模拟执行"
- 11. LoRA:base 冻结,只训低秩 delta

1. RL 核心:去掉数学包装

教材讲 RL 绕来绕去(MDP、贝尔曼、策略梯度定理、重要性采样、GAE...),但内核就一句:采样一批动作 → 算每个动作比平均好多少(advantage) → 好的提高其概率、差的降低(advantage 加权的 log 概率梯度)。让高回报的动作更可能发生。名词是包装,机制极简。

2. 目标 $J(\theta)$ 的完整展开

RL 目标 = 最大化期望回报。以初始状态 s_0 出发:

$$J(\theta) = E_{a \sim \pi}[R] = \sum_a \pi(a|s_0) Q(s_0, a), \quad Q(s_0, a) = \sum_{s'} P(s'|s_0, a) V(s')$$

逐项: $\pi(a|s_0)$ = 策略在 s_0 生成动作 a 的概率; $Q(s_0, a)$ = 采取 a 后到结束的期望回报; $P(s'|s_0, a)$ = 转移概率(采取 a 后到 s'); $V(s')$ = 后续状态的价值。

外层对动作 a 按策略 π 加权,内层是转移 $\times$ 后续价值。

推演往复(转移的稀疏性) — 对很多任务(尤其 LLM 生成),给定 (s_0, a) 后新状态基本**确定**(拼上生成 token 的序列是唯一的)→ 转移 $P(s'|s_0, a)$ 是 one-hot(几乎只有一个 s' 概率为 1)→ $\Sigma_{s'}$ 退化成"确定的下一个状态的 V "。所以 LLM 里转移这层很稀疏(确定性)。

3. 策略梯度:log-derivative trick + 大数定律

想让 J 最大 → 求梯度 ∇J 。但 $J = \Sigma P(a) \cdot R$, ∇P 不好直接用(P 在采样分布里)。用 **log-derivative trick**:

$\nabla P = P \cdot \nabla \log P$ (因为 $\nabla \log P = \nabla P / P$, 两边乘 P)。

代回: $\nabla J = \Sigma P \cdot \nabla \log P \cdot R = \mathbf{E}[R \cdot \nabla \log \pi]$ —— 变成期望,可采样估计。

$$\nabla J(\theta) = \mathbf{E}_{a \sim \pi} [R \cdot \nabla_{\theta} \log \pi(a)] \approx (1/N) \sum_i R_i \nabla_{\theta} \log \pi(a_i)$$

大数定律是根基(教材常跳过):上式的期望 $\mathbf{E}[\dots]$ 靠**采样估计** —— 采 N 个动作,样本均值逼近真实期望(蒙特卡洛)。**两层期望都靠采样**:外层对动作(采 N 个 a_i),内层 V 本身也是期望(采样轨迹的实际回报逼近)。大数定律保证样本均值 → 真实期望。**整个策略梯度 = 蒙特卡洛估计**,这也是为什么 RL 要采很多样本、且方差大难训(采样噪声)。

大白话:想让高回报的动作概率大 → 对每个采样的动作,用回报加权它的 **log 概率梯度** → 回报高的把**概率往上推**,低的往下压。log-derivative trick 把"概率的梯度"变成"能采样的 log 概率梯度"。

2.1 为什么从 $J(\theta)$ 出发

RL 的一切从一个目标开始:**让策略 π (参数 θ)产生的行为,期望回报最大**。形式化就是 $J(\theta) = \mathbf{E}_{a \sim \pi}[R]$ 。训练 = 调 θ 使 J 最大 = 梯度上升 $\theta \leftarrow \theta + lr \cdot \nabla J$ 。所以核心任务是**求 ∇J** ——但 J 是个期望(对策略采样的动作求),期望的梯度不好直接算,这是策略梯度要解决的。

2.2 V 和 Q 的定义(回报的期望)

- $V(s)$ = 从状态 s 出发,按策略 π 走到结束的**期望总回报** = $\mathbf{E}[\sum \gamma^t r_t \mid s_0=s]$ 。
- $Q(s,a)$ = 在 s 先采取动作 a ,之后按 π 走的期望总回报。
- 关系(贝尔曼): $Q(s,a) = r(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s')$ —— 即时 reward + 折扣后续价值。
- $J(\theta) = V(s_0) = \sum_a \pi(a|s_0) Q(s_0,a)$ (对初始状态所有动作按策略加权)。

3.1 log-derivative trick 的完整推导

$J(\theta) = \sum_a \pi_{\theta}(a) R(a)$ (离散;连续换积分)。求梯度:

$$\nabla_{\theta} J = \sum_a \nabla_{\theta} \pi_{\theta}(a) \cdot R(a)$$

问题: $\nabla \pi$ 不是期望形式(不能直接采样)。技巧——乘除 π :

$$\nabla_{\theta} J = \sum_a \pi_{\theta}(a) \cdot [\nabla_{\theta} \pi_{\theta}(a) / \pi_{\theta}(a)] \cdot R(a) = \sum_a \pi_{\theta}(a) \cdot \nabla_{\theta} \log \pi_{\theta}(a) \cdot R(a)$$

(用了 $\nabla \log \pi = \nabla \pi / \pi$)。右边现在是 " π 加权的求和" = **期望**:

$$\nabla_{\theta} J = E_{a \sim \pi} [R(a) \cdot \nabla_{\theta} \log \pi_{\theta}(a)]。$$

为什么这个技巧关键: $\nabla \pi$ 无法采样(它不是"按 π 加权"的形式),但 $\nabla \log \pi$ 乘上 π 就变回"按 π 加权的期望" → 能用采样估计。**这是策略梯度的核心数学**——把"概率的梯度"变成"能采样的 $\log$ 概率梯度期望"。

3.2 大数定律:为什么采样能估期望

$\nabla J = E_{a \sim \pi} [R \cdot \nabla \log \pi]$ 是期望,真实值要对所有可能动作/轨迹积分(算不出)。**大数定律:**独立采 N 个样本 $a_i \sim \pi$,样本均值 $(1/N) \sum R(a_i) \nabla \log \pi(a_i)$ **依概率收敛到真实期望** ($N \rightarrow \infty$)。所以:

$$\nabla J \approx (1/N) \sum_i R(a_i) \nabla \log \pi(a_i)。$$

两层期望都靠采样:① 外层对动作(采 N 个解/轨迹)② 内层 R/V 本身是期望(采样轨迹的实际回报逼近 V 的定义)。

这是蒙特卡洛估计,教材常一笔带过("E $\approx$ (1/N)Σ")但不点破根基是大数定律。后果:① N 要大(样本少方差大)② RL 天生高方差难训(采样噪声)③ advantage 减 baseline、GAE 权衡都是**降方差**的手段(在采样估计的框架内减噪声)。

4. advantage: $A = Q - V$

用回报 R 直接加权方差大 → 减一个 baseline 降方差:用 **advantage $A = Q(s,a) - V(s)$** (这个动作比该状态平均好多少)。

- **$V(s)$** = 从 s 到结束的期望回报(所有动作的平均,baseline)。
- **$Q(s,a)$** = 采取 a 后的期望回报。
- **$A > 0$:**这动作比平均好 → 强化; **$A < 0$:**比平均差 → 抑制。

$$\nabla J = E[A \cdot \nabla \log \pi]。$$

Q 怎么估:不单独估 Q ,用 **$Q(s,a) = r + \gamma V(s')$** (即时 reward + 折扣后续价值,贝尔曼)。所以 $A = r + \gamma V(s') - V(s)$ (TD advantage)。**只需 critic 估 V** (一个网络), Q 借 $V(s')$ 拼出; $V(s')$ 仍要 critic。**GAE** 则是加权多个 n -step TD,权衡偏差(依赖不准的 V)与方差(采样噪声)——是估计,非 exact(真实 V 未知,critic 只近似)。

推演往复(单步 $V(s)$ 的意义) — 单步时 $V(s)$ 不是没意义,是**退化成 baseline(平均 reward)**:单步采取 a 直接得 r (无后续 s'), $Q = r, V(s) =$ 所有动作 reward 的期望 = 平均 → $A = r -$ 平均 reward。

5. Code generation:单步 bandit + GRPO

code generation 的建模:给题 → 生成解 → 测试 → reward → **结束**(无状态转移)。**单步 contextual bandit**:

- **无 state 转移**(state 恒为题目,不切换)→ 不需要多步 MDP 那套。
- **action** = 生成的解(token 序列);用**不同温度采样 N 个解**(多样性来源:温度差异,覆盖低温保守 + 高温探索)。
- **reward** = **partial(通过测试比例,如 8/10)**,比 0/1 信号密——GRPO 组内对比时能区分"过 8 vs 过 5"(0/1 则都是 0 无法对比)。

GRPO(Group Relative Policy Optimization,砍 critic):单步无 s' , $A = r - \text{baseline}$ 。**baseline 不用 critic 网络估,用组内 N 个解的平均 reward**:

$$A_i = (r_i - \text{mean}(r)) / \text{std}(r) \quad (\text{组内归一化,除 std 让尺度可比、训练更稳})$$

组内均值当 baseline = 砍了 PPO 的 critic/value 网络(不用训 V)。这是 GRPO 相比 PPO 的核心简化。

推演往复(建模的三处纠正) — ① 温度是**采样超参**(采不同解的手段),不是 action(解才是 action);② 过/不过是 **reward**(结果),不是 state(state 是题目);③ code generation **单步 bandit,不需要 state 概念**(题目恒定不切换)——多步 state 那套是 debug/SWE-agent 才需要。

6. policy loss = advantage 加权的自生成序列 CE

RL 没有监督 label(不知"正确解")。policy loss:

$$L = - \sum_i A_i \log \pi(\text{解}_i)$$

"label" = **自己采样的解**, "权重" = **advantage**。 $A_i > 0$ 提高这个解的概率, $A_i < 0$ 降低。求导(A 当常数): $\nabla L = - \sum A_i \nabla \log \pi(\text{解}_i)$ 。

两层结构:外层 A_i 对应第 i 个整段解(解级 advantage);内层这段解**还是逐 token 的 CE**($\log \pi(\text{解}) = \sum_t \log P(\text{token}_t | \text{前文})$,自回归,和正常 LLM 训练一样)。合起来: **$A_i \times (\text{这段的逐 token CE}) = \text{解级 advantage} \times \text{token 级 CE}$** 。

CE vs RL(形式像,来源不同):

- CE(监督): $-\log P(\text{外部正确 token})$,系数固定 1。
- RL: $-A \cdot \log P(\text{自己生成的 token})$,系数是 advantage。

形式都是 $\log P$,但 RL 的 token 是自己生成的、系数是 A(不是外部标签、系数 1)。**RL = advantage 加权的、对自生成序列的 CE**。

梯度回传:和 CE 一样,softmax 下**所有输出口都回传**(不只采样的口),整个网络所有层反向传播(embedding 及前面所有参数都更新)。

7. clip 与 KL 锚定(PPO 的 proximal)

在 policy loss 上加两件事:

- **clip(ratio)**:把 $\log \pi$ 换成 ratio $r = \pi_{\text{new}}/\pi_{\text{old}}$,clip 到 $[1-\epsilon, 1+\epsilon]$:

$$L = -\sum \min(r \cdot A, \text{clip}(r, 1-\epsilon, 1+\epsilon) \cdot A)$$

作用:**限制新旧策略别差太远**(Proximal 近端)——防一步更新太大跑偏;且策略梯度基于旧策略采样估计,新策略离太远估计就不准(重要性采样失效),clip 保证在有效邻域内。

- **KL 锚定**: $-\beta \cdot \text{KL}(\pi \parallel \pi_{\text{ref}})$,**ref = SFT/初始模型**(如 Gemma-4-it)。防跑偏、防 reward hacking、不忘原有能力。

8. 框架 vs 算法:RLVR / RLHF / GRPO / PPO

	是什么	区别
RLHF	框架	reward = 人类偏好(reward model 学的,主观,易 reward hacking)
RLVR	框架	reward = 可验证对错(测试/答案,客观,不可 hack)——code 用这个
PPO	算法	带 critic(估 V),clip 近端
GRPO	算法	砍 critic(组内均值当 baseline),clip 近端

框架(RLVR/RLHF)决定 reward 来源;**算法(PPO/GRPO)**决定怎么用 reward 更新。底层数学互通(单步/多步策略梯度 + advantage 加权),只是 reward 建模不同。GRPO/PPO 都可用于 RLVR 或 RLHF。你的 **code RL = RLVR(测试当 reward) + GRPO(砍 critic)**。RLVR 的 reward 客观可信(不像 RLHF 的 RM 有噪声),这是 DeepSeek-R1 能纯 RL 涌现推理的原因。

4.1 baseline 为什么能降方差(不改期望)

用 R 直接加权,R 值波动大 $\rightarrow$ 梯度估计方差大。减一个 baseline b(不依赖动作):

$$\nabla J = E[(R - b) \cdot \nabla \log \pi].$$

关键:减 b 不改变期望(因为 $E[b \cdot \nabla \log \pi] = b \cdot E[\nabla \log \pi] = b \cdot \nabla (\sum \pi) = b \cdot \nabla 1 = 0$,概率和恒为 1 梯度为 0),但**能降方差**(R-b 比 R 波动小)。取 $b = V(s)$ (状态平均价值) $\rightarrow R - V \approx Q - V = \text{advantage}$ 。

所以 advantage = 减了 baseline 的回报,期望不变、方差更小。

4.2 GAE 的偏差-方差权衡

advantage 有多种估法:

- **一步 TD**: $A = r + \gamma V(s') - V(s)$ —— 低方差(只 1 步实际 r)高偏差(依赖不准的 V)。
- **实际多步回报**: $A = \sum \gamma^t r_t - V(s)$ —— 低偏差(实际回报)高方差(整条轨迹采样噪声)。
- **GAE**:用 λ 加权多个 n-step, $A = \sum (\gamma \lambda)^t \delta_t$ ($\delta = \text{TD error}$)。 λ 小偏一步 TD, λ 大偏实际回报。**权衡偏差与方差**。

都是**估计**(真实 V 未知, critic 只近似),非 exact。这就是你 CarRacing 手写过的 GAE。

5.1 单步为何不需要 critic

多步: $A = r + \gamma V(s') - V(s)$, 要 critic 估 V 。单步(bandit): 生成完就结束, 无 s' , $A = r - \text{baseline}$ 。
baseline 只需"平均 reward"——GRPO 直接用组内 N 个解的平均当 baseline, 不用 critic 网络估 V 。这就是单步 + GRPO 能砍 critic 的原因: baseline 有现成的(组内均值), 不必训 value 网络。

5.2 组内归一化的作用

$A_i = (r_i - \text{mean}(r)) / \text{std}(r)$:

- 减 mean: baseline(比组内平均好多少)。
- 除 std: 归一化尺度——不同题的 reward 方差不同(有的题普遍高分、有的低), 除 std 让不同题的 advantage 可比, 梯度尺度稳定, 训练更稳。

组内 N 个解共享一个 mean/std, 每个解算自己的 A_i 。

6.1 policy loss 求导的细节

$L = -\sum_i A_i \log \pi_\theta(\text{解}_i)$ 。求导时 A_i 当常数(detach/stop gradient)—— A 由 reward/组内均值给定, 不对它求导(policy 梯度只调"生成动作的概率")。

$\nabla_\theta L = -\sum_i A_i \nabla_\theta \log \pi(\text{解}_i)$ 。

$\log \pi(\text{解}_i) = \sum_t \log P(\text{token}_t | \text{前文}; \theta)$, 所以 ∇ 对每个 token 的 log 概率求梯度(和 CE 反向传播一样), 乘以解级系数 A_i 。

推演往复(V/critic 的两个角色) — V /critic 有两个独立的角色, 别混: ① 在 critic loss 里被训(MSE: 让 V 逼近实际回报) ② 在 policy loss 里当常数(detach)(A 含 V , 但更新 policy 时不对 A 求导)。PPO 两个 loss 都有(policy + critic + entropy); GRPO 只有 policy loss(A 用组内均值, 无 critic loss)。

推演往复(梯度回传所有口) — 曾以为 RL "只从采样的动作那个输出口回传"。纠正: softmax 下所有输出口都回传(和 CE 一样): 采样动作那口显式($\partial \log \pi / \partial \text{logit}$ 主导), 其余口通过 softmax 归一化间接回传。CE vs RL 的区别不在"哪些口回传"(都全传), 在目标来源(外部 one-hot 正确 token vs 自己采样 + advantage 加权)。且反向传播是整个网络(embedding 及所有层都更新), 不只输出层。

9. Code debug / SWE-agent: 多步 MDP + GAE

debug 相比 code generation 的核心变化: 从单步 bandit 变多步 MDP(有状态转移)。

- state: 当前程序执行状态(输出/报错) + 当前代码 + 题目 + 历史(改过什么)。
- action: 基于 state(报错/输出)生成下一步修改(改代码); SWE-agent 里还含导航(读文件/搜索, 竞赛 9 个工具)。
- 转移: 改代码 → 重新执行 → 新 state(新输出/报错)。
- reward: partial(问题解决程度/测试通过比例/报错减少), 比 0/1 密。
- 多步 MDP(有状态转移, 不是 bandit) → 用 GAE / 累积 return / 信用分配(多轮里哪步修改关键)。

Kaggle SWE-agent 本质就是 code debug:给 repo + issue → 导航 repo → 定位 → 改代码 → 跑测试验证(多轮:定位→改→测→修)。与本节 debug 建模同类。**区别只在规模/工具**:SWE-agent 是"大 repo 版 debug"(多了导航/读文件/搜索工具),本质一样(多步 debug 循环)。本节 debug 建模直接用于 Kaggle:state=repo/报错,action=改代码+导航,reward=测试 partial,多步 MDP(GAE)。

10. state 表示:文本 / 执行图 / 变量值

debug 的 state 编码很关键(高维代码/报错怎么表示,决定 agent 决策)。三层(由易到难):

- **文本类(易)**:代码、报错文字、单元测试失败信息(哪行/期望 vs 实际)→ 直接 embedding(LLM 原生吃)。主流。
- **执行图(可 embedding)**:AST/调用图/数据流图 → **图串行化成文本**(或用 code graph embeddings,竞赛官方提供)。
- **变量值/执行内部(最难)**:动态、无界、高维(变量值任意)。预定义状态(离散化)会丢信息,新建状态会爆炸——没有优雅解,实际靠"关键变量文本快照 + 截断"折中。

推演往复(Kaggle 有无变量值 debug) — Kaggle 的 9 个工具:有代码结构(code graph/neighbors/subgraph)+ run_command(跑测试得报错文本),**没有 debugger/看变量值/单步执行的工具**——action 集合定死,state 靠"代码 + 报错文本",没有"运行时变量值"这层。原因:① action 定死 ② 执行内部状态建模太难(官方避开)。所以竞赛的 SWE-agent 是"看代码+报错"的 debug(文本类 state),不是"看变量值"的深度 debug。

LLM 的"心算模拟执行"(涌现能力):现在强 LLM(Claude/GPT)改代码时会在推理链里"假设输入走一遍代码,变量变成啥,哪里错"——**但这是脑内心算模拟(CoT 推理),不是真执行/真 trace**。能力来自预训练 + 推理 RL(见过海量"代码→执行结果"配对,next-token 逼它学"代码→行为"的泛化映射,R1 式 CoT 强化)。本质仍是 next-token 拟合出的"代码行为世界模型"——对新程序也能推(学到的是泛化规律)。**内部怎么实现:黑盒(不可解释)**,但能力是真的。这与真 trace(外部喂变量值)是两回事:一个模型内部心算,一个外部真执行喂入。

11. LoRA:base 冻结,只训低秩 delta

不改原始权重 W (冻结),对每个矩阵加低秩增量 ΔW :

$$\Delta W = B \cdot A (B: d \times r, A: r \times d, r \text{ 小如 } 16), \text{前向} = X \cdot W + X \cdot (B \cdot A)。$$

delta 不和原 W 一样大:只存 B 、 A (如 $8192 \times 16 + 16 \times 5376 \approx 22$ 万,原 W 4400 万的 $\sim 0.5\%$)。求导只对 B 、 $A \rightarrow$ AdamW 状态只对这 $0.5\% \rightarrow$ 省显存(QLoRA 单卡够)。假设:微调需要的 ΔW 是低秩的(几个方向够)。

SFT 和 RL 都只训 LoRA(base 冻结,梯度只到 LoRA)。符合竞赛规则(base 官方固定,只允许 LoRA adapter)+ 省钱。

7.1 clip 为什么防跑偏(重要性采样视角)

策略梯度用旧策略 π_{old} 采样的数据估计,更新到 π_{new} 。若 π_{new} 离 π_{old} 太远,用旧数据估的梯度就不准了(重要性采样权重 π_{new}/π_{old} 失控)。**clip** 把这个 ratio 夹在 $[1-\epsilon, 1+\epsilon]$, 保证更新在"旧数据估计仍有效"的邻域内。这就是 "Proximal(近端)" 的本质:小步更新,别让新旧差太大(否则采样估计失效 + 训练崩)。

7.2 KL 锚定 ref 的作用

$-\beta \cdot \text{KL}(\pi \parallel \pi_{ref})$, $ref = \text{SFT/初始模型(如 Gemma-4-it)}$ 。三个作用:① **防 reward hacking**(policy 为刷 reward 生成怪东西, KL 拉回)② **不忘原有能力**(离 SFT 太远会丢失语言流畅性/通用能力)③ **稳定训练**(锚定参考,不漂移)。clip(限制新旧步幅) + KL(锚定 ref) 一起构成 proximal 的保护。

9.1 debug 从单步到多步的本质变化

code generation(单步 bandit):给题 $\rightarrow$ 一次生成 $\rightarrow$ reward $\rightarrow$ 结束。**无状态转移**, $V(s)$ 退化成平均。
debug(多步 MDP):改代码 $\rightarrow$ 执行 $\rightarrow$ 新报错 $\rightarrow$ 再改 $\rightarrow \dots$ 。**有状态转移**(代码状态随每次修改变化), 回到完整 RL:累积 return ($\sum \gamma^t r_t$)、 $V(s)$ 有实际意义(从当前代码状态到修好的期望回报)、GAE 做多步信用分配(多轮里哪次修改关键)。

这正是 CarRacing 那套多步 RL(trajectory / GAE / 信用分配)——debug 的多轮 = CarRacing 的多步,框架直接迁移,换环境(赛车 $\rightarrow$ 代码-执行)。

9.2 信用分配(多步的难点)

多轮 debug 里,最后修好了,**哪一次修改是关键?**(可能第 3 次改对了核心,前两次在探索)。这是**信用分配**——把最终 reward 合理分到各步。GAE 用折扣 + TD 做这个(靠后接近成功的步分更多,靠前探索的步分少)。稀疏 reward(只最后成功给分)时信用分配尤其难 $\rightarrow$ 过程 reward(每步的部分改进)能缓解。

11.1 LoRA 为什么低秩够(微调的内在维度)

假设:**微调(适配新任务)需要的权重变化 ΔW , 内在维度低**——不需要满秩大改,几个方向的调整就够(适配任务 $\neq$ 重学能力)。所以 $\Delta W = B \cdot A$ (秩 $\leq r$) 能表达微调需要的变化。这也是为什么强 base + LoRA 有效(能力在 base, LoRA 只做低秩微调对齐);而弱 base(如从零训的小模型)LoRA 救不了(能力不在,不是低秩微调能补的)。

11.2 QLoRA 省显存的账

回顾 Part3:全参训练显存大头是 AdamW 状态(m, v FP32, 占参数量 $\times 8$)。**QLoRA**:base 4bit 量化冻结(无梯度/优化器) + 只有 LoRA($\sim 0.5\%$ 参数)有梯度和 AdamW 状态 $\rightarrow$ 把 " $\times 8$ 的 AdamW 大头" 压到

只对 0.5% 的 LoRA。31B 全参训练 ~400G+, QLoRA ~40-55G, 单卡够。SFT 和 RL 都只训 LoRA(base 冻结), 符合 Kaggle 规则(base 官方固定, 只允许 LoRA adapter)。

小结

RL 训代码模型的完整链: $J(\theta) = E[R] \rightarrow \text{log-derivative trick}(\nabla P = P \nabla \log P) + \text{大数定律(蒙特卡洛采样估期望)} \rightarrow \text{advantage}(A = Q - V \text{ 降方差}) \rightarrow \text{code generation(单步 bandit + GRPO 组内 baseline 砍 critic)} \rightarrow \text{policy loss(advantage 加权的自生成序列 CE, 外层解级 } A \times \text{ 内层 token CE)} \rightarrow \text{clip} + \text{KL 锚定(proximal)} \rightarrow \text{debug/SWE-agent(多步 MDP + GAE)} \rightarrow \text{state 表示(文本/图/变量值)} \rightarrow \text{只训 LoRA(base 冻结)}。$

框架(RLVR)+ 算法(GRPO)+ LoRA + 多步(SWE-agent)—— 正是下一步 Kaggle Gemma-4 SWE-agent 竞赛的全部理论底子。

— Part 4 完。理论打底完成, 下一步进 Kaggle SWE-agent 竞赛工程。 —